

RELATÓRIO DE ANÁLISE DE PROJETO (MILESTONE II)

1. Equipe

Nome: Christian Salles Castilho	E-mail: c.salles@gec.inatel.br	Matr.: 1981
Nome: Giovana Emanuely Fonseca	E-mail: giovana.emanuely@ges.inatel.br	Matr.: 320
Nome: Karolina Oliveira da Silva	E-mail: karolina.oliveira@ges.inatel.br	Matr.: 115
Nome: Maria Clara Ribeiro Ignácio	E-mail: m.clara@gec.inatel.br	Matr.: 1980
Nome: Samuel Almeida Ralise	E-mail: samuel.ralise@gec.inatel.br	Matr.: 1993

2. Perfil do Projeto a ser desenvolvido:

(Será uma aplicação Web, Mobile ou Progressive Web App (PWA)? Explicar a motivação da escolha da equipe mais em detalhes.)

O projeto será uma aplicação Web.

A decisão veio de quem vai usar a plataforma. A proposta da Secretaria lista quatro perfis: gestores da Prefeitura e da SMCELT, empreendedores do setor (hotéis, pousadas, comércio e serviços), pesquisadores e estudantes, e a população. Nenhum desses grupos consulta indicador de turismo todo dia. É uma consulta pontual, feita quando alguém vai planejar uma temporada, escrever um trabalho ou tomar uma decisão de gestão. Ninguém instala aplicativo no celular para abrir duas ou três vezes por ano. Com uma aplicação Web, o usuário chega pelo navegador, sem instalar nada e sem depender de loja de aplicativos.

Outro ponto é o endereço. Os relatórios e indicadores do Observatório vão ser citados em ofício, em e-mail, em matéria de jornal e no site da Prefeitura. Uma página web abre por um link, e um aplicativo nativo não. Como o objetivo do projeto é tornar pública uma informação que hoje fica em arquivo interno, poder mandar o link de um indicador pesa mais do que qualquer recurso de celular.

Pesou também quem mantém a plataforma depois de nós. A proposta prevê que a Secretaria atualize os dados e possa evoluir o sistema ao final do projeto. Com aplicação Web, o painel administrativo abre em qualquer computador da Prefeitura sem instalação, uma correção entra no ar para todos de uma vez, e não existe conta de desenvolvedor paga nem aprovação de loja no caminho. Um aplicativo nativo criaria custo recorrente e dependência técnica que uma secretaria municipal pequena teria dificuldade de sustentar.

Boa parte da população vai acessar pelo telefone, e isso não exige aplicativo. A interface será responsiva desde o começo: o dashboard, as tabelas de indicadores e a lista de relatórios se ajustam ao tamanho da tela. Quem entra pelo celular vê a mesma plataforma, no navegador.

Por fim, a escolha combina com o que foi acertado com o parceiro. A proposta da Secretaria pede uma plataforma web, e nenhuma das oito funcionalidades descritas depende de câmera, GPS ou notificação push. Ir para mobile custaria mais e não entregaria nada a mais.

3. Identificação das possíveis ferramentas a serem utilizadas no desenvolvimento

(Para cada parte do projeto (front, back-end e persistência), apresentar uma descrição das tecnologias que serão usadas, seus pontos fortes e fracos. Sugere-se fortemente uma pesquisa para compreender estes detalhes a serem inseridos)

3.1. FRONT-END: React (com Vite e Recharts)

React é uma biblioteca JavaScript para construir interfaces com componentes reutilizáveis. No nosso projeto, isso permite separar cards, gráficos, filtros e linhas de relatório em partes menores, que podem ser ajustadas e reaproveitadas nas diferentes telas.

Escolhemos React porque a nossa interface se repete muito. Card de indicador, gráfico, filtro e linha de relatório reaparecem em várias telas, e escrever isso como componente economiza trabalho. Pesou também o ecossistema de gráficos: existem bibliotecas prontas para o dashboard, que é a parte central da plataforma, e vamos usar o Recharts. Além disso, duas integrantes, Karolina e Maria, indicaram ter conhecimento de React no Milestone I, e como o backend também é JavaScript, quem terminar sua frente consegue ajudar em outra sem trocar de linguagem. O build e o servidor de desenvolvimento ficam por conta do Vite.

Pontos Fortes:

- Componentes reutilizáveis: um card de indicador ou um gráfico é escrito uma vez e usado em todas as telas, o que economiza trabalho num semestre curto;
- No projeto de teste, poderemos montar gráficos de barra, linha e pizza com o Recharts usando componentes prontos. Isso nos ajudará a confirmar que o dashboard não precisará ser montado do zero;
- Quando testamos a ferramenta, o servidor do Vite subiu em segundos e a tela recarregou sozinha a cada alteração, o que encurta o nosso ciclo de tentativa e erro;
- Mesma linguagem do backend, o que facilita o rodízio de pessoas entre as frentes.

Pontos Fracos:

- React é biblioteca, não framework. Roteamento, formulário e chamada de API vêm de bibliotecas de fora, então o grupo precisa decidir coisas que um framework já entregaria decididas;
- Hooks, estado e código assíncrono têm curva de aprendizado, e dois integrantes apontaram isso como ponto a desenvolver no Milestone I;
- Na instalação vimos que um projeto vazio já baixa centenas de pacotes. Isso nos preocupa pela manutenção depois que a Secretaria assumir a plataforma;
- A tela é montada no navegador, e vimos que atrapalha a indexação por buscadores. Para um site público de prefeitura, que deveria aparecer numa busca por turismo em Santa Rita do Sapucaí, isso é uma limitação.

3.2. BACK-END: Node.js com Express

Node.js é um ambiente de execução JavaScript fora do navegador. Com o Express, ele permite organizar rotas e tratar as requisições e respostas de uma aplicação do lado do servidor.

No nosso projeto, vamos usar o Express para organizar as rotas, os middlewares e o tratamento das requisições e respostas da API.

Escolhemos essa dupla principalmente pela linguagem única no projeto: JavaScript no front e no back barateia a troca de frente durante o semestre, que é um combinado do nosso grupo. A nossa API também é pequena e bem delimitada, com consulta de indicadores por filtro, listagem e download de relatórios, autenticação e escrita pelo painel administrativo, e o Express dá conta desse porte sem impor estrutura pesada. O npm tem pacote pronto para tudo o que precisamos, desde upload de arquivo para os PDFs, autenticação por token para o painel da Secretaria e até o acesso ao PostgreSQL. Além disso, a integrante Maria, com mais conhecimento de backend, disse ter experiência em Node.js e desenvolvimento de APIs no Milestone I.

<u>Pontos Fortes:</u> - Uma só linguagem no projeto inteiro, o que permite ao grupo circular entre as frentes sem reaprender sintaxe; - Subir a API de teste levou dois comandos e um arquivo de dez linhas. Começar rápido conta para nós, que temos seis semanas de implementação; - Procuramos no npm e existe pacote conhecido para o que precisamos, upload dos PDFs e login do painel, então não vamos escrever isso do zero; - O exemplo rodou igual nas máquinas do grupo, considerando cada uma das configurações, o que importa porque somos cinco com máquinas diferentes.	<u>Pontos Fracos:</u> - O Express não impõe organização. Sem disciplina, o projeto degenera em um arquivo de rotas gigante. Vamos definir a estrutura de pastas ainda no Milestone IV para evitar isso; - JavaScript não tem tipagem estática, então erro de tipo só aparece na execução. É um risco real numa API que trata série histórica, e vamos avaliar se adotamos TypeScript posteriormente; - Ao montar alguns exemplos, encontramos tutoriais feitos para versões diferentes do Express, e isso confundiu quem estava configurando o projeto; - Segurança fica por conta de quem desenvolve: cabeçalhos, validação de entrada e proteção do painel administrativo precisam ser implementados, não vêm por padrão.
--	--

3.3.PERSISTÊNCIA: PostgreSQL (com Prisma)

PostgreSQL é um sistema gerenciador de banco de dados objeto-relacional de código aberto. No nosso projeto, ele armazenará indicadores, períodos, setores e relatórios relacionados entre si.

Os dados do Observatório são tabulares e ligados entre si: um indicador pertence a um período e a um setor da economia turística, e um relatório pertence a um ano de publicação. Banco relacional é o que atende esse formato. A plataforma também precisa responder perguntas do tipo quantos leitos existiam no segundo semestre de 2025 no setor de hospedagem, e em SQL isso é uma consulta de agregação com filtro por intervalo, que é onde a tecnologia se sai melhor. Pesou ainda ser gratuito e sem licença comercial, porque a Secretaria assume a plataforma ao final, e um dos integrantes ter declarado conhecimento de PostgreSQL e Prisma no Milestone I. O Prisma faz a ponte entre o Node e o banco.

<u>Pontos Fortes:</u> - Integridade e transações (ACID): um dado publicado não se perde nem fica pela metade. Isso conversa com o objetivo da proposta de preservar o histórico do Observatório entre mudanças de gestão; - Montando o modelo do banco percebemos que quase toda consulta do dashboard é soma por período e por setor, e é isso que SQL faz melhor. Foi o que fechou a escolha contra usar algo mais simples; - Chaves estrangeiras e restrições impedem cadastrar um indicador sem setor válido, o que protege a base de erro de digitação no painel administrativo; - Licença livre e sem custo de licença comercial. Para nós, isso reduziria a dependência de	<u>Pontos Fracos:</u> - Foi o item que mais deu trabalho no nosso setup. Parte do grupo travou na instalação, com a senha do usuário postgres e o psql fora do path, e por isso adotamos a imagem Docker para padronizar; - Exige modelagem cuidadosa antes de programar, e mudar a estrutura depois dá trabalho, com migrações. Ou seja, o modelo entidade-relacionamento do Milestone III precisa ser bem feito; - Ajuste fino de desempenho, como criar índice e ler plano de execução, exige conhecimento que a equipe ainda não tem, e as consultas de série histórica são as mais sensíveis a isso; - A equipe declarou, no Milestone I, pouca experiência prática com PostgreSQL em projeto de maior complexidade, então parte do
--	---

orçamento vindo da Secretaria para manter a plataforma depois de uma possível entrega;	semestre vai ser aprendendo e o cronograma precisa de folga para isso.
--	--

4. Instalação e Comprovação do Setup

(Anexar aqui prints que comprovem a instalação/configuração das ferramentas escolhidas para desenvolvimento do projeto)

Roteiro de instalação executado pela equipe

Cada integrante executou os passos abaixo na própria máquina. Os prints de comprovação estão inseridos em seguida, identificados por integrante.

1. Node.js (versão 22 LTS ou superior)

Instalado a partir de nodejs.org

2. Git

Instalado a partir de git-scm.com.

3. PostgreSQL 18

Instalado a partir de postgresql.org/download ou via imagem docker.

4. Docker Desktop

Instalado a partir de docker.com.

5. Projeto React com Vite

Criado com `npm create vite@latest` usando o template React.

6. Projeto Node com Express

Criado com `npm init -y` seguido de `npm install express`.

7. Repositório no GitHub

Repositório do grupo criado e com os cinco integrantes como colaboradores.

8. Figma

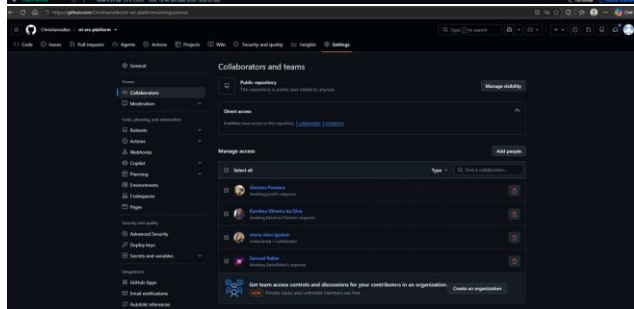
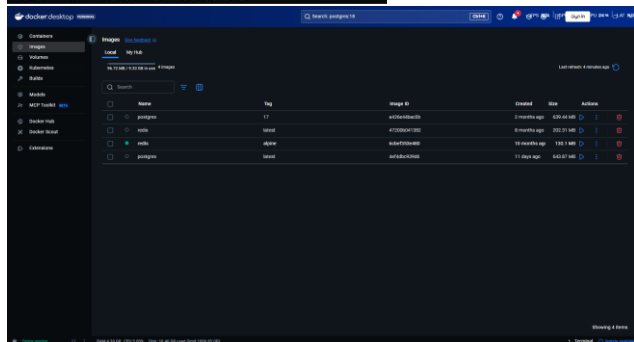
Conta criada.

Christian:

```
C:\Users\chris>node -v
v24.12.0
```

```
C:\Users\chris>npm -v
11.18.0
```

```
C:\Users\chris>git --version
git version 2.55.0.windows.5
```



```
PS C:\Users\chris\projetos\meu-react> npm run dev
> meu-react@0.0.0 dev
> vite

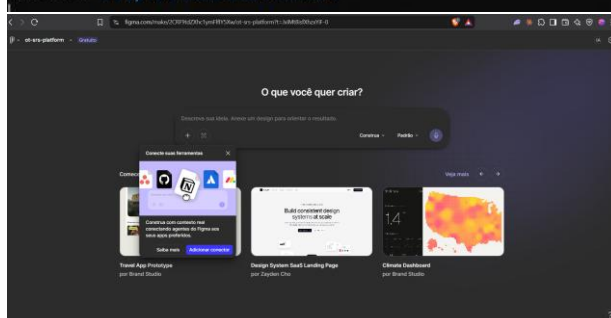
VITE v5.2.11 ready in 491 ms

→ Local:    http://localhost:5173/
→ Network:  use --host to expose
→ press h + enter to show help

✓ Compiled successfully in 285ms (147 modules)
11:32:45 AM [vite] (client) hmr update /src/App.jsx
```

```
PS C:\Users\chris\projetos\meu-express> node index.js
```

Servidor Express rodando!
Acesse: <http://localhost:3000>



Maria Clara:

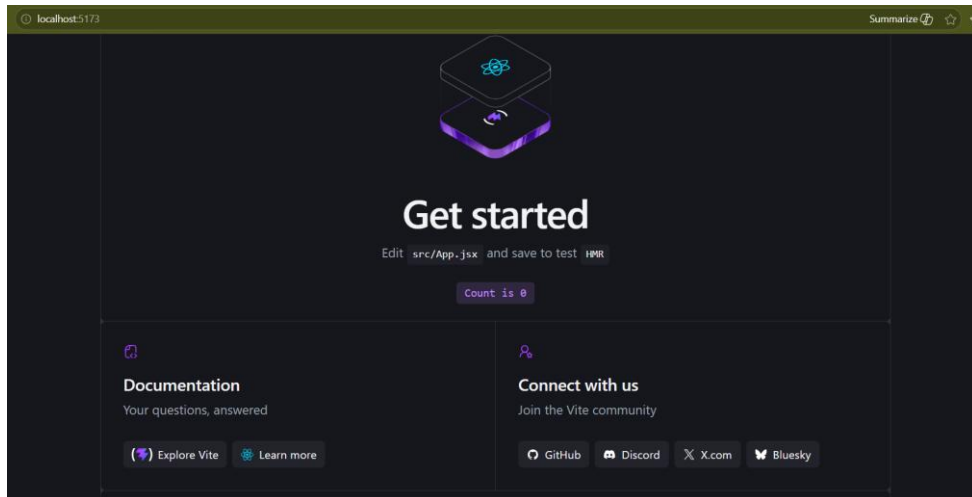
```
PS C:\Users\mclar\Documents\Projetos\inatel_p9> node -v
v24.14.0
PS C:\Users\mclar\Documents\Projetos\inatel_p9> npm -v
11.9.0
PS C:\Users\mclar\Documents\Projetos\inatel_p9> █
```

```
PS C:\Users\mclar\Documents\Projetos\inatel_p9> git --version
git version 2.49.0.windows.1
PS C:\Users\mclar\Documents\Projetos\inatel_p9> █
```

```
git version 2.49.0.windows.1
PS C:\Users\mclar\Documents\Projetos\inatel_p9> winget search PostgreSQL
PostgreSQL 18.6 (Debian 18.6-1.pgdg13+2) on x86_64-pc-linux-gnu, compiled by gcc (Debian 14.2.0-19) 14.2.0, 64-bit
```

```
PS C:\Users\mclar\Documents\Projetos\inatel_p9> docker exec -it postgres18 psql -U postgres -d aula -c "SELECT version();"
          version
-----
PostgreSQL 18.6 (Debian 18.6-1.pgdg13+2) on x86_64-pc-linux-gnu, compiled by gcc (Debian 14.2.0-19) 14.2.0, 64-bit
(1 row)
PS C:\Users\mclar\Documents\Projetos\inatel_p9> █
```

```
PS C:\Users\mclar\Documents\Projetos\inatel_p9> npm create vite@latest projeto-react -- --template react
◇ Scaffolding project in C:\Users\mclar\Documents\Projetos\inatel_p9\projeto-react...
◇ Installing dependencies with npm...
added 24 packages, and audited 25 packages in 26s
9 packages are looking for funding
  run `npm fund` for details
found 0 vulnerabilities
◇ Starting dev server...
> projeto-react@0.0.0 dev
> vite
VITE v8.2.2 ready in 2030 ms
→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
█
```



```
PS C:\Users\mclar\Documents\Projetos\inatel_p9> mkdir projeto-express

Directory: C:\Users\mclar\Documents\Projetos\inatel_p9

Mode                LastWriteTime         Length Name
----                -
d-----          04/09/2026   17:21                projeto-express

PS C:\Users\mclar\Documents\Projetos\inatel_p9> cd projeto-express
PS C:\Users\mclar\Documents\Projetos\inatel_p9\projeto-express> npm init -y
Wrote to C:\Users\mclar\Documents\Projetos\inatel_p9\projeto-express\package.json:

{
  "name": "projeto-express",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}

PS C:\Users\mclar\Documents\Projetos\inatel_p9\projeto-express> npm install express

added 68 packages, and audited 69 packages in 3m

27 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\mclar\Documents\Projetos\inatel_p9\projeto-express>
```

```
PS C:\Users\mclar\Documents\Projetos\inatel_p9\projeto-express> notepad server.js
PS C:\Users\mclar\Documents\Projetos\inatel_p9\projeto-express> node server.js
Servidor rodando em http://localhost:3000
```

Giovana:

```
PS C:\Users\gioem> node -v
v22.20.0
```

```
PS C:\Users\gioem> npm -v
11.6.4
```

```
PS C:\Users\gioem> git --version
git version 2.38.1.windows.1
```

```
PS C:\Users\gioem> winget search PostgreSQL
```

A fonte 'msstore' requer que você exiba os seguintes contratos antes de usar.

Terms of Transaction: <https://aka.ms/microsoft-store-terms-of-transaction>

A origem requer que a região geográfica de duas letras do computador atual seja enviada ao serviço de back-end para funcionar corretamente (por exemplo, "EUA").

Você concorda com todos os termos dos contratos de origem?

[Y] Sim [N] Não: y

Nome	ID	Versão	Correspondência	Origem
PostgreSQL 17	PostgreSQL.PostgreSQL.17	17.11-3	Moniker: postgresql	winget
PostgreSQL 18	PostgreSQL.PostgreSQL.18	18.6-3	Moniker: postgresql	winget
Postgres Pro Standard 13	PostgresPro.Standard.13	13.23	Moniker: postgresql	winget
Postgres Pro Standard 14	PostgresPro.Standard.14	14.20	Moniker: postgresql	winget
Postgres Pro Standard 15	PostgresPro.Standard.15	15.15	Moniker: postgresql	winget
Postgres Pro Standard 17	PostgresPro.Standard.17	17.7	Moniker: postgresql	winget
Hangar	Anssollabs.Hangar	1.0.0	Tag: postgresql	winget
Benthic Software: GoldSquall	BenthicSoftware.GoldSquall.1	1.5.0.130	Tag: postgresql	winget
Benthic Software: GoldSquall	BenthicSoftware.GoldSquall.2	2.4.0.244	Tag: postgresql	winget
DBVisualizer	DBVis.DBVisualizer	26.2.1	Tag: postgresql	winget
DBeaver	DBeaver.DBeaver.Community	26.2.0	Tag: postgresql	winget
DBeaverEE	DBeaver.DBeaver.Enterprise	26.1.0	Tag: postgresql	winget
DBeaverLite	DBeaver.DBeaver.Lite	26.1.0	Tag: postgresql	winget
DBeaverTeam	DBeaver.DBeaver.Team	26.1.0	Tag: postgresql	winget
DBeaverUltimate	DBeaver.DBeaver.Ultimate	26.1.0	Tag: postgresql	winget
tabularis	Debba.Tabularis	0.22.0	Tag: postgresql	winget
HeidiSQL	HeidiSQL.HeidiSQL	12.21.0.7344	Tag: postgresql	winget
Luna Modeler	Ideamerit.LunaModeler	14.0.1	Tag: postgresql	winget
DbGate	JanProchazka.dbgate	7.2.6	Tag: postgresql	winget
Laragon	LeNgockhoa.Laragon	8.7.0	Tag: postgresql	winget
LibreDB Studio	LibreDB.Studio	0.13.7	Tag: postgresql	winget
pgAdmin 4	PostgreSQL.pgAdmin	9.17	Tag: postgresql	winget
psqlodbc	PostgreSQL.psqlODBC	18.00.0002	Tag: postgresql	winget
psqlodbc (mimalloc version)	PostgreSQL.psqlODBC.mimalloc	18.00.0002	Tag: postgresql	winget
Navicat Premium	PremiumSoft.NavicatPremium	17.3.11	Tag: postgresql	winget
Goose	Pressly.Goose	3.27.3	Tag: postgresql	winget
Devaron	Revoltz.Devaron	0.0.1	Tag: postgresql	winget
Keon CLI	Seiraiyu.Keon	0.1.56	Tag: postgresql	winget

Launchpad 0 0 0 Connect

Containers

Images

Volumes

Builds NEW

Dev Environments BETA

Docker Scout

Extensions

Add Extensions

Containers

Give feedback

Container CPU usage 0.00% / 800% (8 CPUs available)

Container memory usage 0B / 7.51GB

Show charts

Search

Only show running containers

	Name	Image	Status	CPU (%)	Port(s)	Last started	Actions
☐	> nutriplan		Exited	0%		9 months ago	
☐	> s204		Running (4/4)	0%		1 minute ago	

Showing 2 items

Multi-container applications

8 mins

Containerize your application

3 mins

View more in the Learning center

Engine running

RAM 2.62 GB CPU 12.75%

Not signed in

New version available 2

```
PS C:\Users\gioem> npm create vite@latest
|
| Select a framework:
| React
|
| Select a variant:
| TypeScript + React Compiler
|
| Which linter to use?
| Oxcint
|
| Install with npm and start now?
| Yes
|
| Scaffolding project in C:\Users\gioem\vite-project...
|
| Installing dependencies with npm...
|
added 71 packages, and audited 72 packages in 24s
13 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
|
| Starting dev server...
|
> vite-project@0.0.0 dev
> vite

VITE v8.2.2 ready in 12545 ms
→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

```
PS C:\Users\gioem> npm init -y
"@shelex/cypress-allure-plugin": "^2.41.2",
"@types/react": "^19.1.11",
"cypress-plugin-tab": "^1.0.5",
"mochawesome": "^7.1.4"
},
"name": "gioem",
"version": "1.0.0",
"description": "",
"main": "cypress.config.js",
"scripts": {
  "test": "mocha"
},
"keywords": [],
"author": "",
"license": "ISC",
"type": "commonjs"
}
```

● PS C:\Users\gioem> npm install express

added 2 packages, removed 1 package, changed 13 packages, and audited 342 packages in 12s

61 packages are looking for funding
run `npm fund` for details

18 vulnerabilities (1 low, 5 moderate, 11 high, 1 critical)

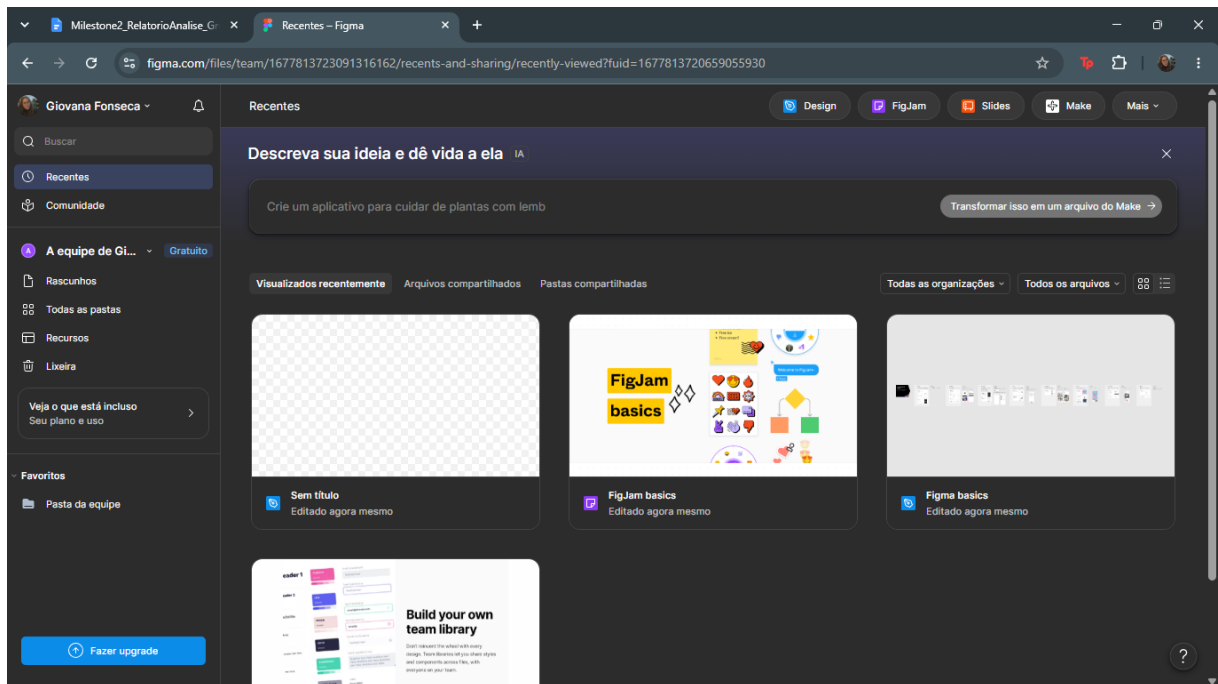
To address issues that do not require attention, run:
`npm audit fix`

To address all issues (including breaking changes), run:
`npm audit fix --force`

Run `npm audit` for details.

❖ PS C:\Users\gioem>

The screenshot shows the GitHub repository settings page for 'ot-sr-platform' by user 'Christianalles'. The 'Collaborators and teams' tab is selected. The page indicates it is a 'Public repository'. Under 'Direct access', it shows 4 entities with access: 1 collaborator and 3 invitations. The 'Manage access' section lists four collaborators: Giovana Fonseca, Karolina Oliveira da Silva, maria clara ignacio, and Samuel Ralhe, each with a 'Remove' button. A sidebar on the left contains navigation links for General, Access, Moderation, Code, Actions, Webhooks, Copilot, Planning, Environments, Code Spaces, Pages, Security and quality, Advanced Security, Deploy keys, Secrets and variables, and Integrations. The bottom of the page promotes creating an organization.

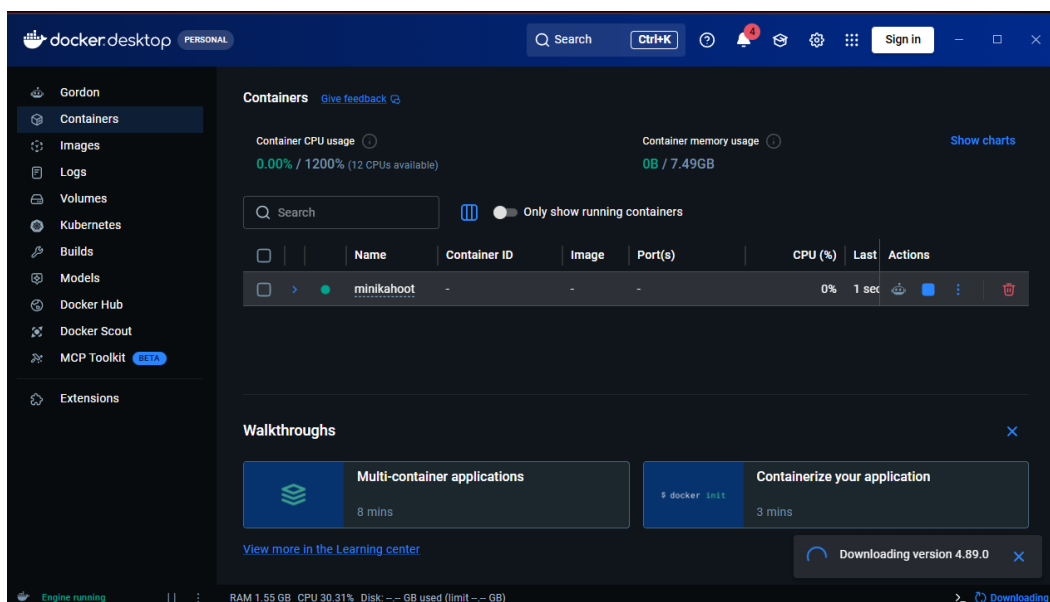


Samuel:

```
Microsoft Windows [Versão 10.0.26200.9168]
(c) Microsoft Corporation. Todos os direitos reservados.

C:\Users\samur>git --version
git version 2.45.1.windows.1

C:\Users\samur>
```



```

PS C:\Users\samur\projeto-react> npm run dev
projeto-react@0.0.0 dev
vite

VITE v5.2.11 ready in 491 ms
  → Local:    http://localhost:5173/
  → Network: use --host to expose
  → press h + enter to show help

C:\Users\samur>psql --version
psql (PostgreSQL) 18.6

C:\Users\samur>

```

```

PS C:\Users\samur\projeto-react> npm run dev
> projeto-react@0.0.0 dev
> vite

VITE v5.2.11 ready in 491 ms
  → Local:    http://localhost:5173/
  → Network: use --host to expose
  → press h + enter to show help

```

```

PS C:\Users\samur\projeto-express> npm init -y
Wrote to C:\Users\samur\projeto-express\package.json:
{
  "name": "projeto-express",
  "version": "1.0.0",
  "main": "server.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  }
}

PS C:\Users\samur\projeto-express> npm install express
added 68 packages, and audited 69 packages in 3m

27 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\samur\projeto-express> node server.js
Servidor rodando em http://localhost:3000

```

Karolina:

```

karol@CTIC-054699NB:~/docker/postgres$ node --version
v24.20.0
karol@CTIC-054699NB:~/docker/postgres$ npm --version
11.19.0
karol@CTIC-054699NB:~/docker/postgres$ |

```

```

karol@CTIC-054699NB:~/docker/postgres$ git --version
git version 2.43.0

```

```

karol@CTIC-054699NB:~/docker/postgres$ docker exec -it postgres psql -U postgres -d meu_banco
psql (18.6 (Debian 18.6-1.pgdg13+2))
Type "help" for help.

meu_banco=# SELECT version();

          version
-----
PostgreSQL 18.6 (Debian 18.6-1.pgdg13+2) on x86_64-pc-linux-gnu, compiled by gcc (Debian 14.2.0-19) 14.2.0, 64-bit
(1 row)

meu_banco=#

```

```
karol@CTIC-054699NB:~/Projeto/frontend/projeto-react$ npm run dev

> projeto-react@0.0.0 dev
> vite

VITE v8.2.2 ready in 104 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help
```

```
karol@CTIC-054699NB:~/Projeto/backend$ nano server.js
karol@CTIC-054699NB:~/Projeto/backend$
karol@CTIC-054699NB:~/Projeto/backend$ node server.js
Servidor rodando em http://localhost:3000
```

Caso utilizem tecnologias não categorizadas acima, listar a partir deste ponto:

Prisma (ORM)

No nosso teste, usamos o Prisma para ligar o código JavaScript ao PostgreSQL e registrar as mudanças do banco em arquivos de migração versionados no Git. Isso facilitou o setup, mas percebemos que o SQL gerado fica menos visível; por isso, as agregações mais pesadas do dashboard poderão ser escritas em SQL direto.

Docker e Docker Compose

No nosso setup, o Docker e o Docker Compose subiram PostgreSQL, back-end e front-end em containers com um comando só, da mesma forma nas cinco máquinas da equipe. Isso ajuda a reproduzir as entregas assíncronas, mas observamos maior consumo de memória e a necessidade de manter a virtualização habilitada, o que pode pesar em um notebook mais fraco, menos potente.

Vite

No projeto de teste, usamos o Vite para criar e executar o front-end em React. Ele iniciou em segundos e atualizou a tela quase na hora a cada alteração, mas acrescenta mais uma ferramenta para quem ainda não conhece o fluxo.

Recharts

No projeto de teste, usamos o Recharts nos gráficos de barra, linha e pizza do dashboard. A integração com os componentes React foi direta, mas percebemos que uma personalização visual

mais específica pode exigir mais trabalho; além disso, ninguém do grupo havia usado uma biblioteca de gráficos antes.

Git e GitHub

Controle de versão e hospedagem do repositório, com revisão por pull request antes de integrar à branch principal, conforme combinamos no Milestone I. O histórico mostra quem fez o quê, o que também serve de evidência de participação individual.

Figma

Usado para desenhar os wireframes das telas, entrega do Milestone III, antes de escrever código de interface. Deixa a gente discutir a navegação com o grupo e com a Secretaria enquanto mudar é barato. O risco é o resultado final se afastar do desenho se ninguém revisar.

Jest

Pretendemos usar o Jest no Milestone VII para testar as regras da API e as consultas de indicadores, tanto no back-end quanto no front-end. Como nenhum integrante tem prática com testes automatizados, essa frente vai exigir estudo e tempo reservado no cronograma.

REFERÊNCIAS

[1] REACT. Início rápido. Disponível em: <https://pt-br.react.dev/learn>

[2] MDN WEB DOCS. Introdução ao Express/Node. Disponível em: https://developer.mozilla.org/pt-BR/docs/Learn_web_development/Extensions/Server-side/Express_Nodejs/Introduction

[3] POSTGRESQL WIKI. Introdução e histórico. Disponível em: https://wiki.postgresql.org/wiki/Introdu%C3%A7%C3%A3o_e_Hist%C3%B3rico

Acesso a todas as referências em setembro de 2026.